

Agent-Oriented Architecture

A practical field guide for designing and governing agent systems

One pattern, four passes

Pass	Session job	What changes
1. See the shape	Make AOA visible through small concrete demos	Small workflow
2. Name the shape	Inspect the parts, boundaries, contracts, and evidence	Running system
3. Recognise the shape	Read frameworks, standards, and platforms through AOA	Real landscape
4. Apply the shape	Design for governability, then choose a contextual next move	Governed estate

This is the take-away version of the course: the key visual from each session, the argument it supports, and a compact set of questions to use in your own estate. The runnable examples and lab instructions stay in the companion `aoa-course` repository.

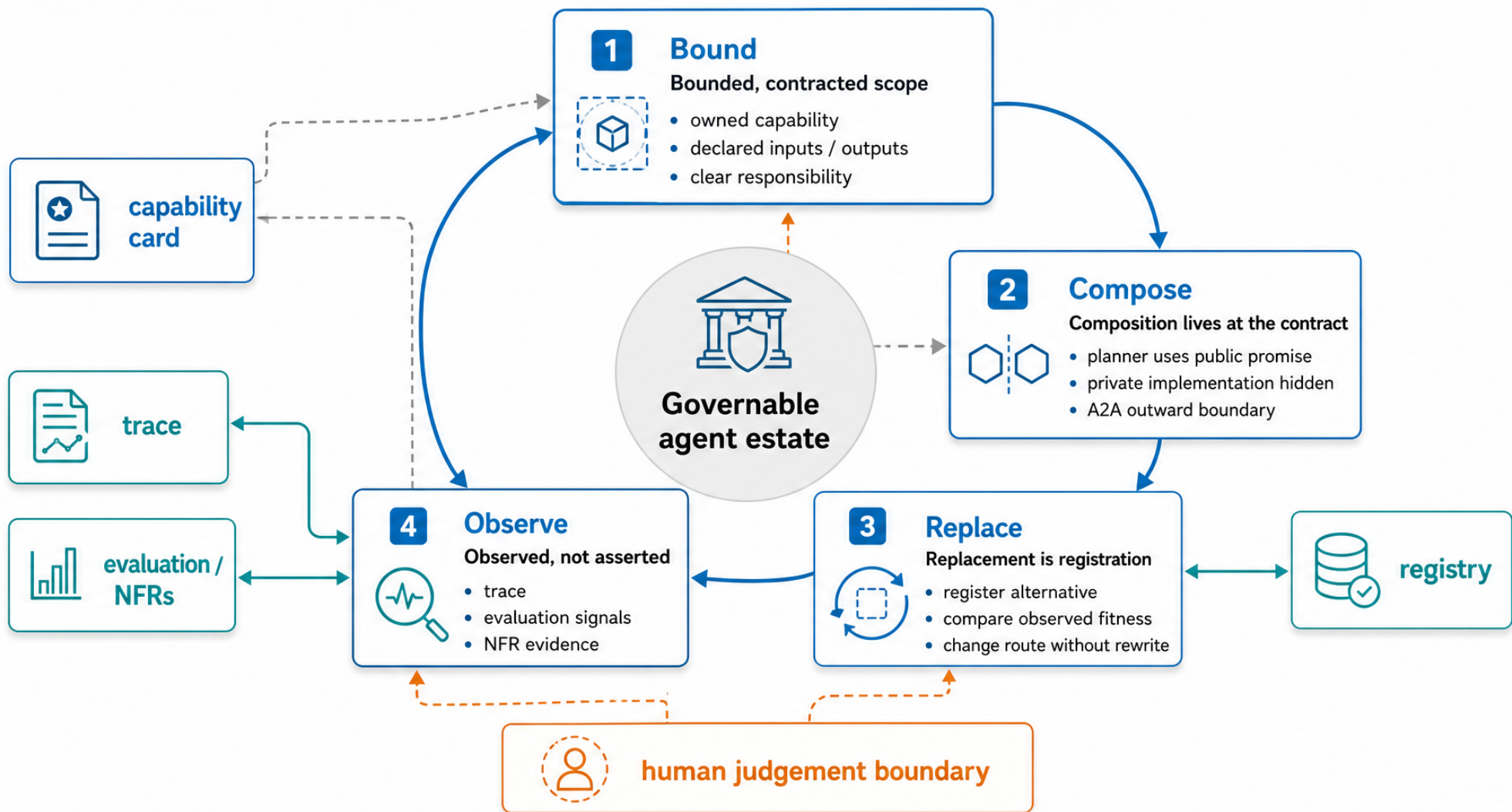
The lifecycle, eight verbs

`describe -> discover -> compose -> execute -> observe -> evaluate -> replace -> govern`

Same eight verbs in every session. Different depth each pass.

The four principles

`bounded, contracted scope · composition lives at the contract · replacement is registration · observed, not asserted`



Principles are useful when they change architecture decisions.

The four AOA principles

The load-bearing architectural commitments. They are constraints on the shape of the system, not features of a product.

1. Bounded, contracted scope

Each Agentic Unit owns a narrow capability with declared inputs, outputs, constraints, and responsibility - a public contract on a capability card. Scope is bounded; an unbounded AU is not an AU.

Test: can you name the decisions this unit hides and its one dominant reason to change?

2. Composition lives at the contract

Work is composed by reading public capability cards through the registry, not by hard-wiring one agent to another. The planner reasons over declared promises and constraints.

Test: could another implementation honour the same contract without changing its consumers?

3. Replacement is registration, not a rewrite

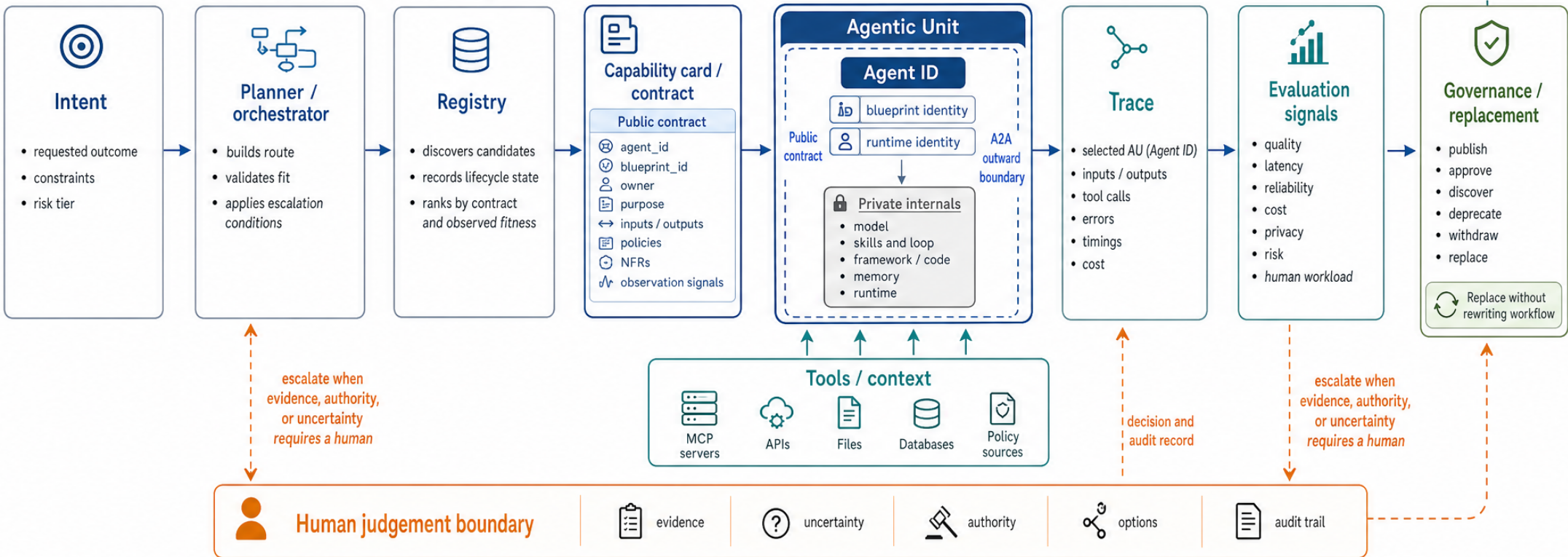
Substituting one AU for another - a different backend, model, vendor, or implementation - is a registry and lifecycle operation. Consumers still depend on the public promise.

Test: what would actually have to change if this model, framework, or supplier disappeared?

4. Observed, not asserted

A capability card is a claim; the trace is evidence. Once traffic flows, runtime observations should qualify or challenge declarations. Selection uses what the estate has measured, not only what a unit claims.

Test: what evidence would falsify the card's claims about quality, cost, latency, or safety?



Governability comes from Agent ID, stable contracts, observable behaviour, explicit judgement boundaries, and replaceable capabilities.

Session 1 · See the shape

Make the fleet-level problem visible, then name the architecture that addresses it.

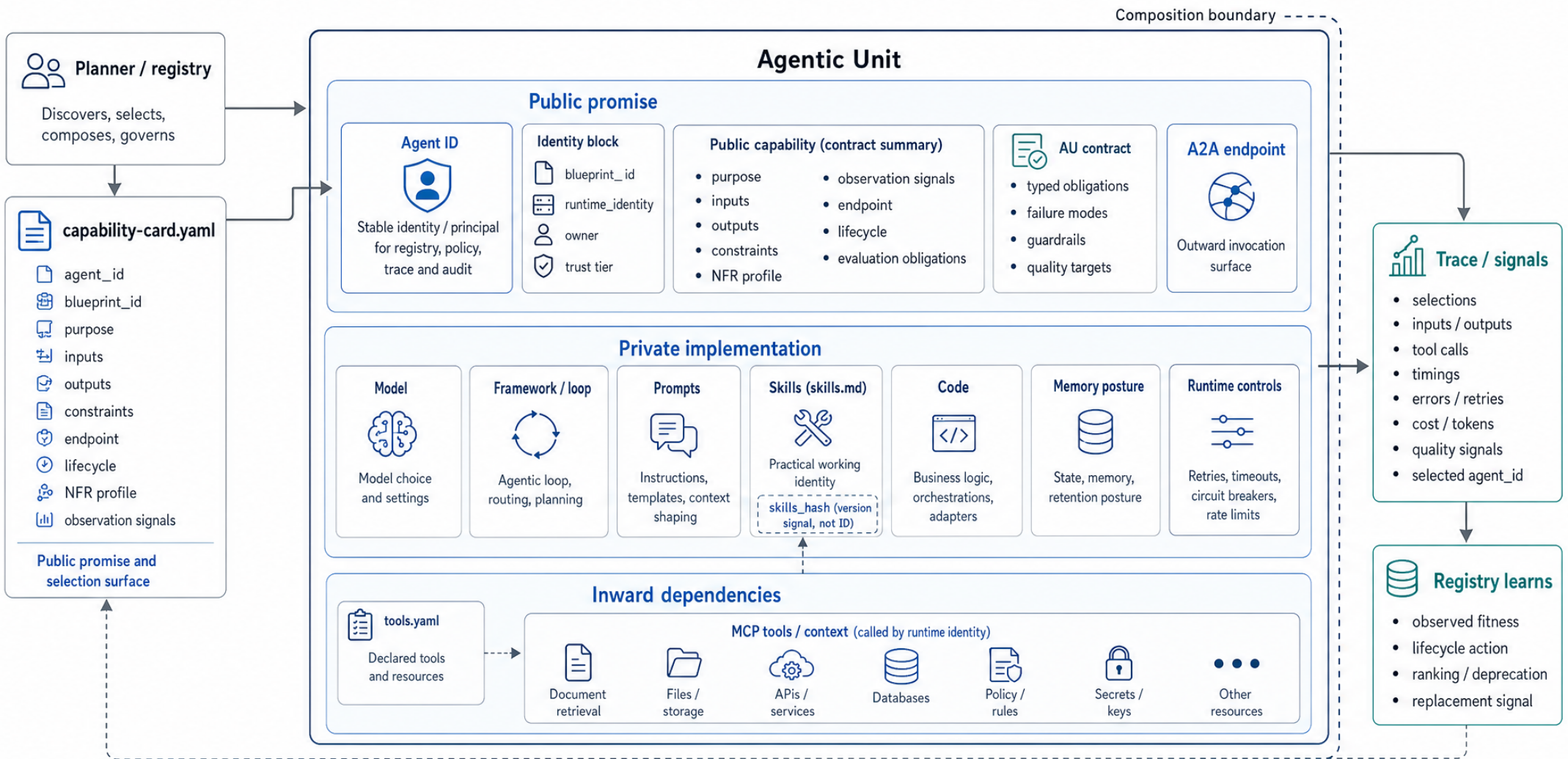
The argument

- **Local acceleration is not system throughput.** Making one agent cleverer does not make a changing estate understandable, dependable, or governable.
- **The agentic inversion changes where composition lives.** A planner can interpret intent and choose capabilities at runtime. That increases adaptability, but also makes identity, declared authority, and evidence essential.
- **Agent sprawl is the fleet-level failure mode.** Without shared control surfaces, duplicate capabilities accumulate, ownership drifts, and shadow agents become invisible dependencies.
- **The unit of value is the estate, not the hero agent.** AOA asks whether capabilities can be described, discovered, composed, observed, evaluated, replaced, and governed together.

The eight control surfaces

Component	Architectural job
Agentic Unit	Own one bounded capability and its decisions
Capability card	Declare the public promise, constraints, and authority
Registry	Make capabilities discoverable and lifecycle-controlled
Planner	Propose a composition from intent and contracts
Tool boundary	Declare where a unit uses external tools or data
Trace	Reconstruct selection, action, output, and escalation
Evaluation signal	Turn quality, cost, latency, and safety into evidence
Human oversight point	Place judgement, approval, or escalation in the path

The expense-report example is deliberately small. Its job is to make the whole shape visible: bounded units, contract-based selection, observed execution, human oversight, and a replaceable model behind the same public capability.



Replace internals without rewriting callers

The AU owns a bounded capability with Agent ID; callers compose against the public promise, not the implementation.

Session 2 · Name the shape

Move from seeing the pattern to inspecting its boundaries, contracts, identities, and evidence.

Public outside, private inside

An AU's **public surface** is the capability card: stable Agent ID, purpose, inputs, outputs, constraints, owner, lifecycle state, non-functional promises, evidence expectations, and oversight policy. Consumers and planners depend on this surface.

Its **private interior** can contain a model, framework, code, instructions, memory, worker agents, and tool calls. Those choices may change without forcing consumers to change. Hashes connect observations to the version that produced them; they are integrity signals, not the unit's identity.

Responsibility must be walkable

intent -> planner -> registry -> selected AU -> tools -> output -> evaluation -> oversight

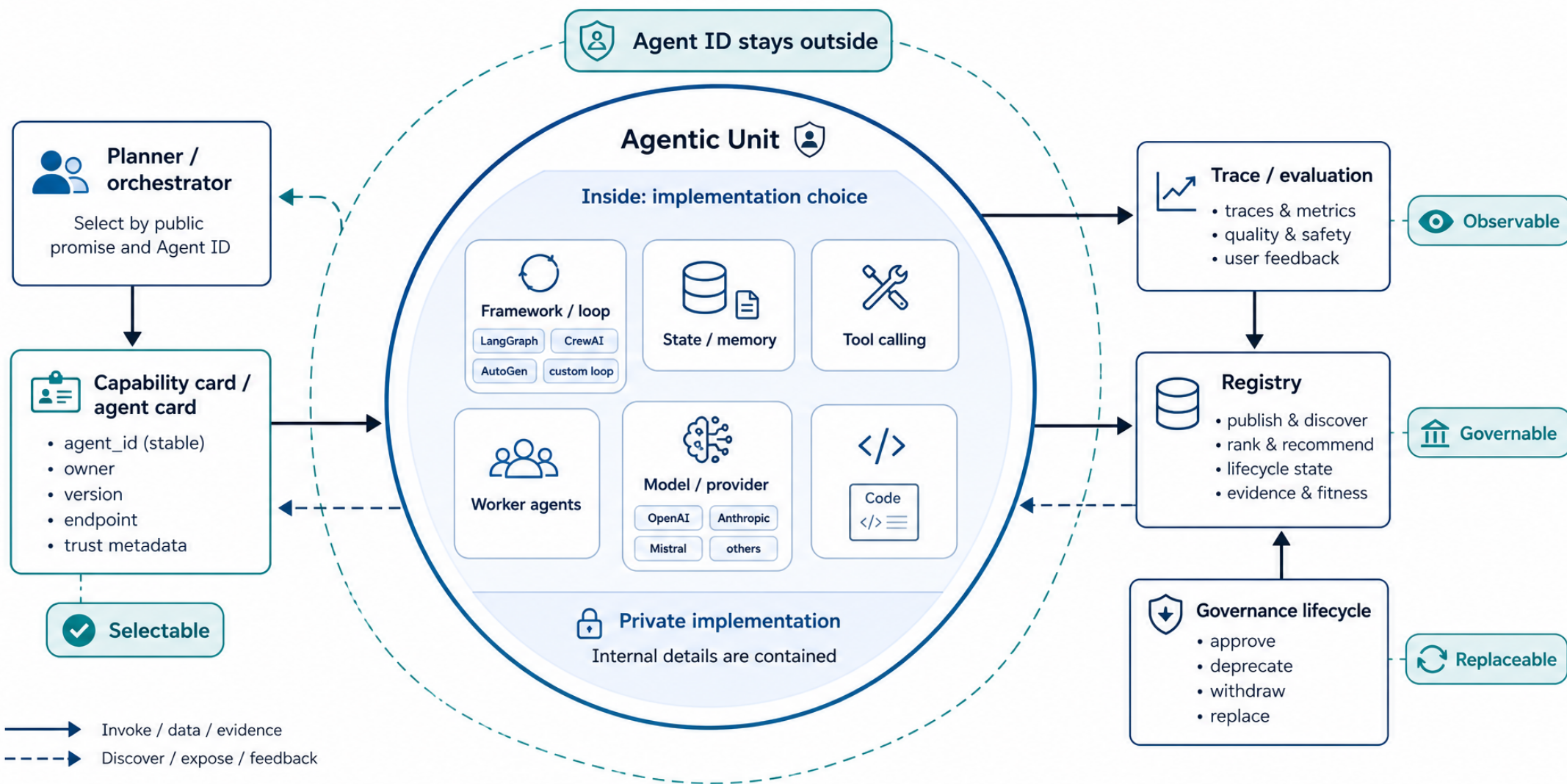
A correlated trace makes that walk inspectable. Silent recovery is acceptable only inside the declared contract and visible in the trace. Missing capability, trust-tier breach, or an irreversible side effect should cross an explicit escalation boundary.

One shape, two domains

The combined Session 1 and 2 lab used the same estate first for CV/job-description evaluation and then for wiki ingestion and query. The important result was not the user interface or the prompts. It was that the registry and traces exposed:

- distinct public capabilities and governed identities;
- shared implementation where sharing was appropriate;
- different promises over the same evaluator identity where that boundary was intentional;
- the same responsibility walk across two domains.

That is the practical test of the architecture: reuse is visible and governed, rather than hidden inside code or a framework.



Frameworks help build the capability. **Agent ID and contracts** let the organisation use and replace it.

Session 3 · Recognise the shape

Use the AOA lens to read frameworks, standards, platforms, economics, and sovereignty without confusing them.

Read the landscape by responsibility

What you are looking at	AOA question
Orchestration framework	What bounded public contract does this workflow expose outside itself?
Personal or maximalist agent	Who owns its authority, memory, lifecycle, and risk?
AOA fleet	What is discoverable, observable, governable, and replaceable across the estate?
A2A	How is an agent capability exposed and invoked across a boundary?
MCP	How does a unit reach tools and context inside its boundary?
Platform	Which useful responsibilities and patterns can we adopt without inheriting its boundary?

The lineage supplies both patterns and warnings

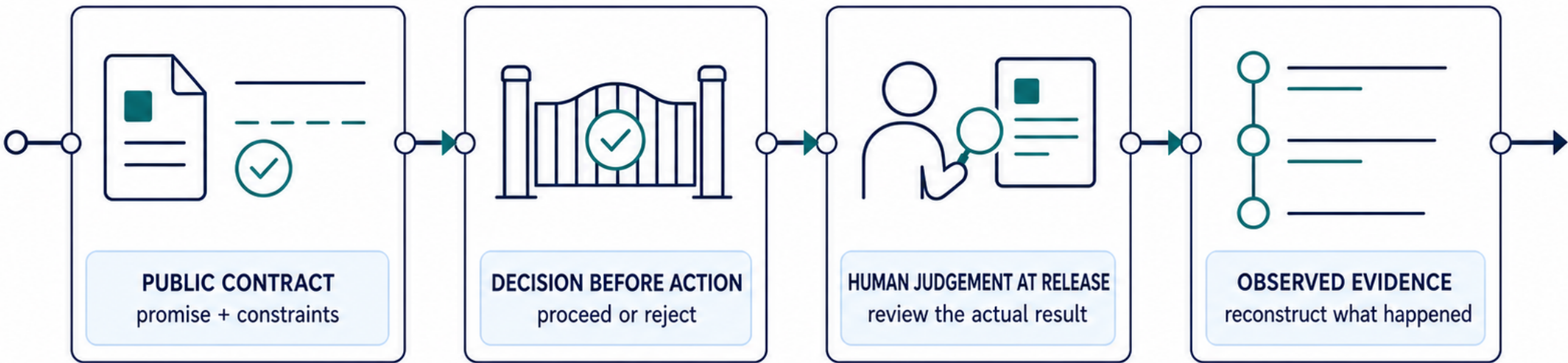
AOA continues lessons from object orientation, SOA, REST, microservices, events, and cloud-native systems. The UDDI warning still matters: a catalogue of stale self-asserted claims is worse than no catalogue. A useful registry therefore owns publish, approve, discover, observe, deprecate, replace, and withdraw - with runtime evidence attached.

The retail example and the EVE lab make the same point from opposite directions. A real platform architecture already contains valuable agent patterns; a strong agent runtime can be an excellent AU interior. Neither is automatically the public architectural boundary. **Framework inside; contract outside.**

Real-world constraints belong in the architecture

- **Tokenomics:** frontier capability has a different cost curve from smaller or local models, while access to the newest models is increasingly tiered. Model supply, price, rate limits, and access can all interrupt a capability. Register alternatives and select with observed evidence rather than assuming uniform access.
- **Sovereignty:** workload location matters, including geographic data-processing requirements, but it is only one layer. Ask separately about control of data, model, infrastructure, and national policy. Provider-hosted memory can affect several layers at once.

The closing trust test is: **who is acting, on whose authority, and why should another team or organisation trust it?**



LEGAL ACCOUNTABILITY REMAINS EXTERNAL

GOVERNABILITY IS DESIGNED INTO THE PATH – NOT ADDED AFTER THE OUTPUT

Session 4 · Apply the shape

Design governability into the work path, then choose a bounded and contextual next move.

Applying AOA starts with governance

A public contract states the promise and constraints. A deterministic decision point can reject an impermissible plan before action. Human judgement can inspect the actual result before release. A correlated trace preserves the evidence needed to reconstruct what happened. Legal accountability remains outside the system.

The focused employment example uses EU AI Act Annex III and Article 14 to create concrete design pressure. Its claims are deliberately narrow:

- a card check can show that a required declaration was observed;
- a flow audit can show that required trace evidence was observed;
- a human can approve or reject the exact held result before release;
- none of those facts classifies a real deployment, proves effective oversight, establishes compliance, or grants permission to use it.

The lab demonstrates control surfaces. It is evidence for the architecture, not a production blueprint or an adoption roadmap.

Apply the discipline contextually

1. Pick a **slice** of work and inventory what already exists.
2. Draw each AU **boundary** around one hidden design decision and one dominant reason to change; write the public contract before the prompt.
3. Classify the capability on two axes: context-bound or context-free, judgement or transformation. That changes the right testing, evaluation, protocol, and governance weight.
4. Translate the four principles into local rules.
5. Profile the slice across separate maturity dimensions - never average them into a reassuring score.
6. Find the weak dimension required by the next move: the active constraint.
7. Choose a Now / Next / Later move through five filters: **Value, Readiness, Control, Learning, Reuse**. Reassess when the constraint moves.

Put stronger controls at decision and release boundaries where context, judgement, or consequence demand them. Keep genuinely context-free utility capabilities lightweight and replaceable.

Monday morning

Start with one bounded slice, not an enterprise transformation.

The seven-step loop in one sentence

Pick a slice. Inventory what exists. Draw the capability boundaries. Localise the principles. Profile maturity. Find the active constraint. Make the next move. Reassess when the constraint moves.

The six artefacts of a governable agent estate

1. **AI and agent inventory** - what exists, who owns it, what it touches, and its lifecycle state.
2. **AU candidate list** - what to formalise, merge, retire, or keep experimental.
3. **Local principles map** - generic AOA principles plus your context become local adoption rules.
4. **Maturity profile** - dimensions scored separately, with evidence and disagreement captured.
5. **Constraint-led roadmap** - Now / Next / Later, filtered by Value, Readiness, Control, Learning, and Reuse.
6. **Governance cadence** - owners review evidence, incidents, value, principle alignment, and whether the constraint has moved.

Boundary cues

- Name the capability in the language of the business.
- Ask what design decision it hides and who is entitled to ignore that decision.
- Look for one dominant reason to change; unrelated reasons usually imply multiple units.
- Write the contract before the prompt. The prompt is private; the contract is public.

The honest closing claim

An AOA adoption roadmap should make the organisation more honest before it makes it more automated. It should reveal what capability already exists, where ownership is unclear, which work should be redesigned, which principles are non-negotiable, and which gaps make orchestration unsafe.

Where to go next

- **The running system:** the `aoa-course` companion repository contains the current labs, data, and run instructions.
- **Contact:** `lewis.crawford@equalexperts.com`
- **LinkedIn:** [linkedin.com/in/lewiscrawford](https://www.linkedin.com/in/lewiscrawford)

Compact reference

Core vocabulary

Term	Working definition
Agentic Unit (AU)	A bounded, independently governable capability with a public contract and model-driven judgement inside
Capability card	The public promise: purpose, I/O, constraints, evidence expectations, provenance, and lifecycle metadata
Agent ID	Stable governed identity of the acting unit; a hash is an integrity signal, not identity
Registry	Discovery plus lifecycle: publish, review, approve, observe, deprecate, replace, withdraw
Planner	Proposes a composition by reading intent and capability contracts
Orchestrator	Executes the plan, carries state, enforces policy, and records the trace
Tool	A callable function or service used behind a declared boundary, commonly through MCP
Trace	Correlated runtime evidence across selection, invocation, tools, outputs, signals, and escalation
Evaluation signal	A measurable observation that can challenge the card's claims
Oversight point	A declared boundary where human judgement, approval, or escalation is required

Questions to take into an architecture review

1. What is this unit's one capability and one dominant reason to change?
2. Which decisions and authority does it own, and which are explicitly denied?
3. What do consumers depend on that is independent of framework and model?
4. Who may publish, approve, invoke, observe, deprecate, and replace it?
5. Which tools and data may it access, on whose behalf?
6. What evidence would falsify its quality, cost, latency, or safety claims?
7. Where are silent recovery and explicit escalation each allowed?
8. Can the trace identify the human, agent, model, tools, and policy involved?
9. Can another implementation honour the same public contract without consumer changes?
10. What external legal, risk, or organisational judgement cannot be delegated to the architecture?

The closing test

Can you describe, discover, compose, execute, observe, evaluate, replace, and govern the capability without knowing which framework is inside it?

If not, the architecture boundary is still inside the implementation.